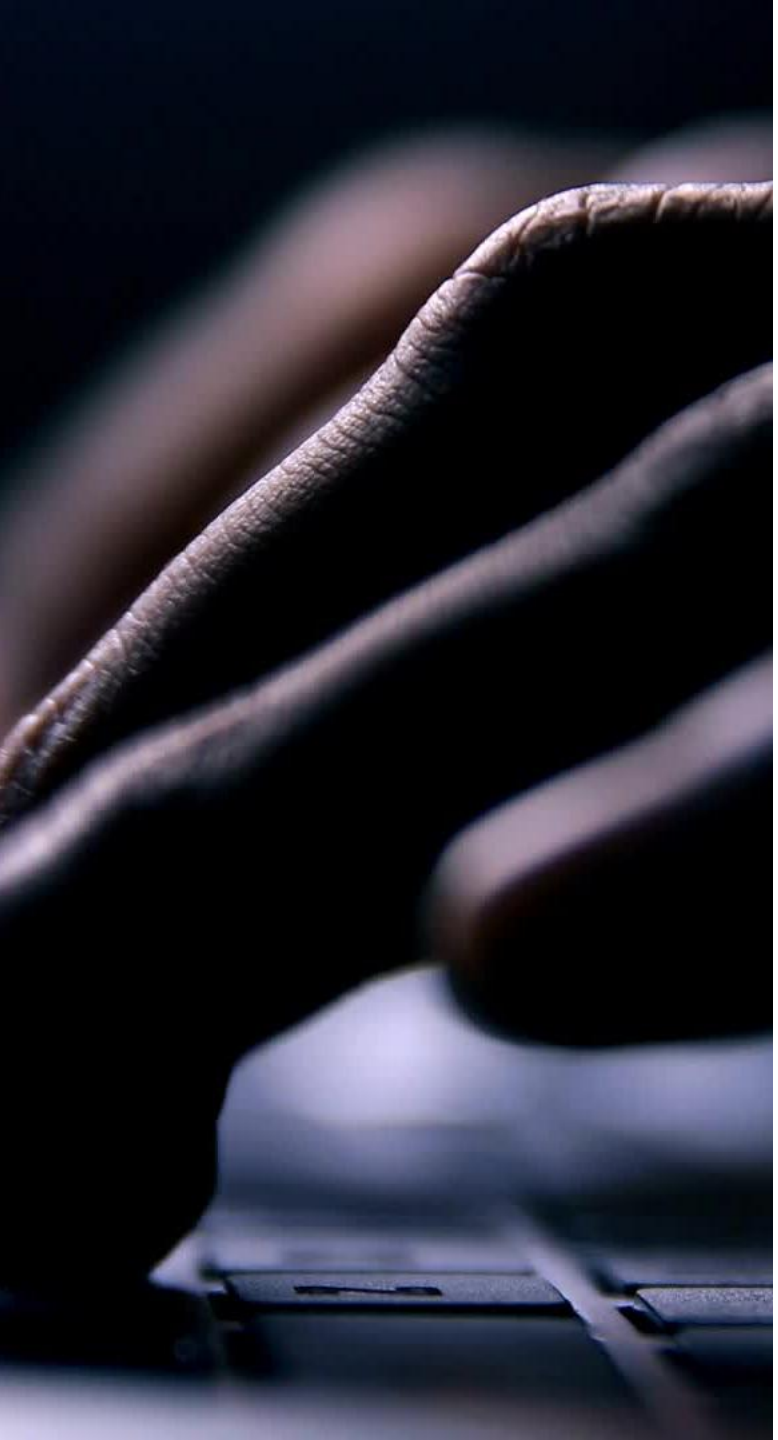


Fatec
Diadema
Luigi Papaiz

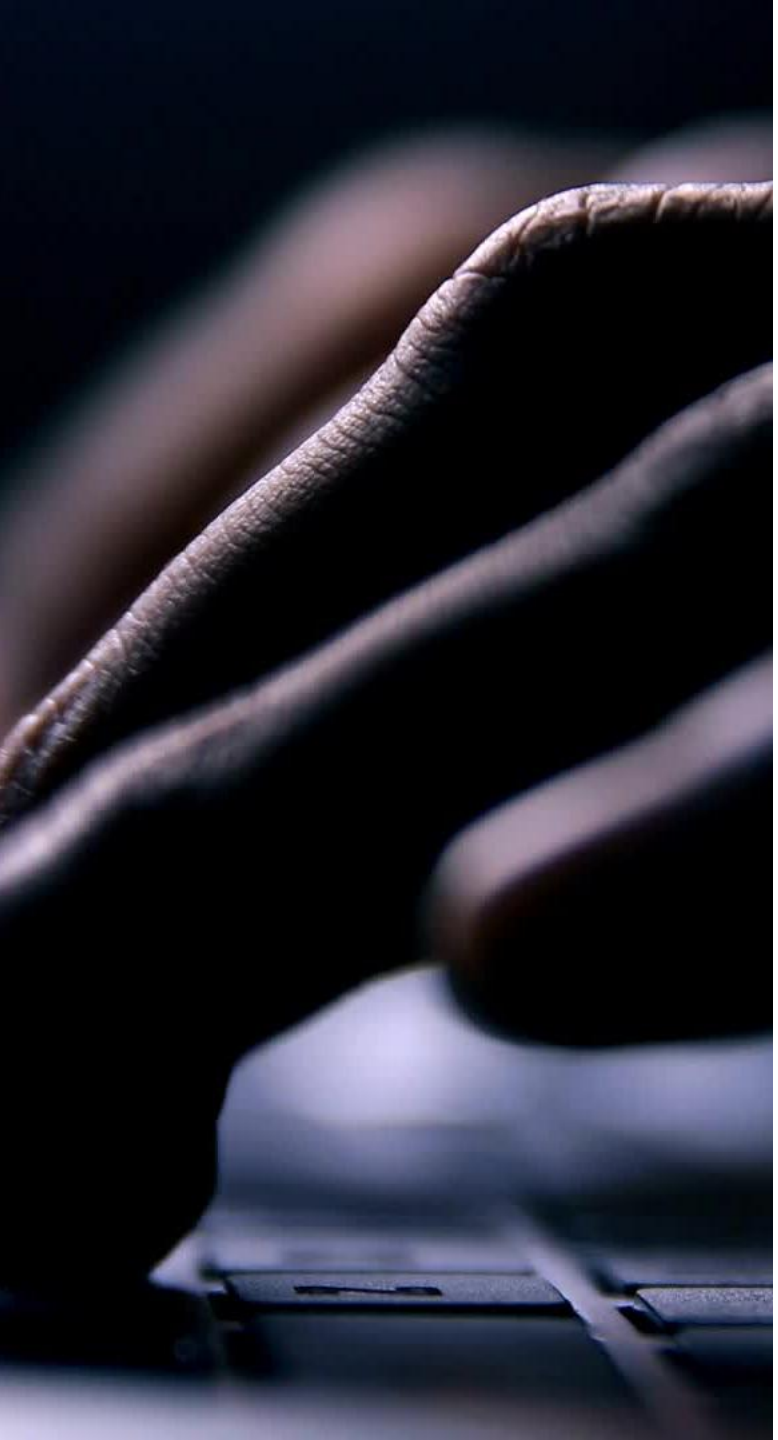
Programação para Dispositivos Móveis I

Prof. Rafael Miranda
rafael.miranda@cps.sp.gov.br

Na última aula

Fatec
Diadema
Luigi Papaiz

- Composable e Preview



Fatec
Diadema
Luigi Papaiz

Comunicação entre Composables

Organizando a Lista de Compras

Vamos separar a tela em partes:

- ListaProdutos → produtos cadastrados
- Resumo → total da compra
- Formulario → entrada de dados

Comunicação entre Composables

Um Composable pode receber informações por meio dos parâmetros da função.

```
@Composable
fun ListaProdutos(
    produtos: List<Produto>
) {
    ...
}
```

Na ListaCompras:

```
ListaProdutos(produtos)
```

O dado sai da ListaCompras e é recebido por ListaProdutos.

ListaProdutos

Responsabilidade: exibir os produtos.

```
@Composable
fun ListaProdutos(
    produtos: List<Produto>
) {
    for (produto in produtos) {
        Text(
            "${produto.nome} - " +
            "${produto.quantidade} x R$ ${produto.preco} = " +
            "R$ ${produto.calcularTotal()}"
        )
    }
}
```

Resumo

O Resumo precisa receber apenas o total.

```
@Composable
fun Resumo(total: Double) {
    Text(
        "Total: R$ $total"
    )
}
```

Na ListaCompras:

```
Resumo(total)
```

Cada Composable recebe somente o que precisa.

E quando precisamos executar uma ação?

O usuário precisa:

- alterar nome
- alterar preco
- alterar quantidade
- executar o cadastro

Os valores continuam pertencendo à ListaCompras:

```
var nome by remember { mutableStateOf("") }
var preco by remember { mutableStateOf("") }
var quantidade by remember { mutableStateOf("") }
```

Formulario recebe os dados

A ListaCompras passa os valores:

```
Formulario(
    nome = nome,
    preco = preco,
    quantidade = quantidade,
    ...
)
```

O Formulário recebe:

```
@Composable
fun Formulario(
    nome: String,
    preco: String,
    quantidade: String,
    ...
){
    ...
}
```

Passando uma ação

O Formulario recebe nome, mas o estado nome pertence à ListaCompras. Por isso, o Formulario recebe uma função para avisar quando o valor mudar.

```
onNomeChange: (String) -> Unit
```

Essa função representa:

- **"Quando o valor mudar, execute uma ação."**

O mesmo acontece com:

```
onPrecoChange: (String) -> Unit
onQuantidadeChange: (String) -> Unit
```

TextField

O TextField recebe o valor e a ação

```
TextField(
    value = nome,
    onValueChange = onNomeChange
)
```

usuário digita
↓
onValueChange
↓
onNomeChange
↓
nome = novo valor

Usando it

Na ListaCompras:

```
Formulario(
    nome = nome,
    onNomeChange = {
        nome = it
    }
)
```

it representa o novo valor digitado.

O mesmo conceito será usado para:

- preco
- quantidade

Botão Cadastrar


```
Button(
    onClick = onCadastrar
) {
    Text("Cadastrar")
}
```

O Formulario não faz o cadastro.

Ele apenas executa a ação recebida.

O cadastro continua na ListaCompras

Fatec
Diadema
Luigi Papaiz

```
Formulario(
    ...
    onCadastrar = {
        val produto = Produto(
            nome = nome,
            preco = preco.toDouble(),
            quantidade = quantidade.toInt()
        )

        produtos.add(produto)

        total += produto.calcularTotal()
    }
)
```

A lógica continua na ListaCompras.

Formulario


```
@Composable
fun Formulario(
    nome: String,
    preco: String,
    quantidade: String,

    onNomeChange: (String) -> Unit,
    onPrecoChange: (String) -> Unit,
    onQuantidadeChange: (String) -> Unit,

    onCadastrar: () -> Unit
) {
    TextField(
        value = nome,
        onValueChange = onNomeChange
    )

    TextField(
        value = preco,
        onValueChange = onPrecoChange
    )

    TextField(
        value = quantidade,
        onValueChange = onQuantidadeChange
    )

    Button(
        onClick = onCadastrar
    ) {
        Text("Cadastrar")
    }
}
```

ListaCompras


```
Column {

    Text("Lista de Compras")

    Formulario(
        nome = nome,
        preco = preco,
        quantidade = quantidade,

        onNomeChange = { nome = it },
        onPrecoChange = { preco = it },
        onQuantidadeChange = { quantidade = it },

        onCadastrar = {
            val produto = Produto(
                nome = nome,
                preco = preco.toDouble(),
                quantidade = quantidade.toInt()
            )

            produtos.add(produto)
            total += produto.calcularTotal()
        }
    )

    ListaProdutos(produtos)

    Resumo(total)
}
```